Build, CI & Deployment Cheatsheet

Lessons: 60 — Building & Packaging · 61 — CI with GitHub Actions · 62 — Deployment & Operations

Examples: 60 · 61 · 62

Covers: build flags, embedding, Docker, GitHub Actions, Kubernetes, signals, rollouts

Legend: `[*]` = flag or feature the lessons have not covered yet

BUILDING THE ARTIFACT

```
go build -o bin/app ./cmd/api        the basic build
CGO_ENABLED=0                        a FULLY STATIC binary — required for scratch
GOOS=linux GOARCH=amd64              cross-compile; no toolchain to install
GOARCH=arm64                         Apple Silicon and Graviton
-ldflags "-s -w"                     strip the symbol table and DWARF (~30% smaller)
-trimpath                            remove local filesystem paths — reproducible builds
-tags=integration,prod               build tags
-race                                the race detector (dev/CI only, ~10x slower)
go tool dist list                [*] every valid GOOS/GOARCH pair
(one static binary with no runtime, no interpreter, no shared libraries — this is
 why Go deployment is simple)
```

VERSION STAMPING

```
var version = "dev"                  a package variable...
go build -ldflags "-X main.version=$(git describe --tags)"    ...set at link time
-X 'main.buildTime=2026-08-30'       one -X per variable; the path must be exact
runtime/debug.ReadBuildInfo()        the toolchain already knows a lot:
  info.Main.Version                  the module version
  info.Settings                      vcs.revision, vcs.time, vcs.modified
expose it                            GET /version -> {"version":..., "commit":...}
(the first question in every incident is "which build is running?")
```

//go:embed

```
import _ "embed"                     required even for the string/[]byte forms

//go:embed version.txt
var version string                   a single file as a string

//go:embed migrations/*.sql
var migrations embed.FS              a whole tree as a filesystem

//go:embed static
var static embed.FS
http.Handle("/static/", http.FileServerFS(static))     serve it directly
rules                                no leading /, no .. , no symlinks; files must
                                     be in or below the package directory
why                                  templates, migrations, and assets ship INSIDE
                                     the binary — one file to deploy
```

MULTI-STAGE DOCKERFILE

```
FROM golang:1.26 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download                  # cached until the deps change
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -trimpath -o /app ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /app /app
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

```
layer order                  go.mod first, source last — the dependency layer caches
.dockerignore                .git, bin, node_modules; it shrinks the build context
scratch                      truly empty: no CA certs, no tzdata, no /etc/passwd
distroless static            scratch + CA certs + tzdata + a nonroot user — the default
alpine                       has a shell for debugging, and musl's DNS quirks
CA certificates              needed for ANY outbound HTTPS call from scratch
exec form ENTRYPOINT         ["/app"] — the shell form makes your process PID 2 and
                             swallows signals
digest pinning           [*] FROM image@sha256:... for reproducibility
buildx --platform        [*] multi-arch images in one push
final image                  ~650KB for a real service with distroless static
```

GITHUB ACTIONS

```
name: ci
on:
  push: { branches: [main] }
  pull_request:
permissions:
  contents: read                     # least privilege, always
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true           # kill superseded runs
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        go: ['1.25', '1.26']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: '${{ matrix.go }}', cache: true }
      - run: go vet ./...
      - run: go test -race -coverprofile=cover.out ./...
```

```
needs: [test]                the job dependency graph
services:                    a real Postgres container for integration tests
  postgres: { image: postgres:16, env: {...}, options: --health-cmd pg_isready }
actions/cache@v4             manual caching when setup-go's isn't enough:
                             key on hashFiles('**/go.sum'), with a restore-keys prefix
actions/upload-artifact@v4   binaries, coverage, reports
secrets.GITHUB_TOKEN         auto-provided; scope it with permissions:
GHCR push                    docker/login-action + build-push-action to ghcr.io
golangci-lint-action     [*] the aggregate linter
govulncheck                  known CVEs — fail the build on them
GoReleaser               [*] on a tag: build the matrix, changelog, GitHub release
Dependabot               [*] .github/dependabot.yml for gomod and actions
pin actions               [*] to a SHA, not a tag, for supply-chain safety
```

A MAKEFILE (one entry point for humans and CI)

```
.PHONY: build test lint docker      declare every non-file target
build:  go build -ldflags "$(LDFLAGS)" -o bin/app ./cmd/api
test:   go test -race -cover ./...
lint:   gofmt -l . && go vet ./... && golangci-lint run
docker: docker build -t $(IMAGE):$(VERSION) .
VERSION := $(shell git describe --tags --always --dirty)
LDFLAGS := -s -w -X main.version=$(VERSION)
TAB, not spaces              make's one unforgivable syntax rule
why                          CI runs the same command you do; nothing drifts
```

THE CI COMMAND SET

```
gofmt -l .                   must print NOTHING; fail the build if it doesn't
go vet ./...
golangci-lint run
go test -race -coverprofile=cover.out ./...
go tool cover -func=cover.out    | the coverage gate
govulncheck ./...
go build ./...                   catches non-test compilation breaks
(run them in that order — the cheap checks fail first)
```

MAKING THE APP OPERABLE

```
12-factor config             environment variables, validated at startup, fail fast
graceful shutdown            signal.NotifyContext(SIGINT, SIGTERM) -> srv.Shutdown(ctx)
                             (see the HTTP sheet for the full shape)
liveness /healthz            am I alive? no dependencies, always cheap
readiness /readyz            can I serve? check the DB; return 503 while draining
flip readyz FIRST            then shut down: the LB stops sending before you stop
                             accepting, so no request is dropped
GOMAXPROCS                   set it to the CPU limit — Go otherwise sees the HOST's
                             cores and oversubscribes the cgroup
GOMEMLIMIT                   set it to ~80% of the memory limit, or the OOM killer
                             wins before the GC tries
log to stdout                the platform collects it; never write log files
```

DOCKER COMPOSE

```
services.api.build / image
depends_on: { db: { condition: service_healthy } }    depends_on alone is NOT ready
healthcheck: { test: [...], interval, timeout, retries, start_period }
restart: unless-stopped
stop_grace_period: 30s       must exceed your drain timeout
deploy.resources.limits      cpus/memory
environment / env_file
docker compose config        validate the file before you ship it
```

KUBERNETES

```
Deployment                   replicas, the pod template, the rolling-update strategy
Service                      a stable virtual IP + DNS name in front of the pods
Ingress                      HTTP routing from outside the cluster
ConfigMap / Secret           non-secret / secret env values
livenessProbe                httpGet /healthz — a failure RESTARTS the container
readinessProbe               httpGet /readyz  — a failure removes it from the Service
startupProbe             [*] for slow starters, so liveness doesn't kill them first
resources.requests           what the scheduler reserves (and the HPA measures)
resources.limits             the hard cap; CPU limits throttle, memory limits OOM-kill
strategy.rollingUpdate       maxSurge / maxUnavailable
terminationGracePeriodSeconds: 30      must exceed your drain time
lifecycle.preStop: sleep 5   let the endpoints propagate BEFORE you stop accepting
                             — this is what actually makes rollouts zero-downtime
securityContext              runAsNonRoot, readOnlyRootFilesystem, drop ALL caps
HPA                          autoscale on CPU or a custom metric
kubectl rollout status/undo  watch it; roll it back
```

OTHER TARGETS

```
PaaS (Cloud Run / Fly.io)    give it a container; it handles TLS, scaling, rollout
                             — read PORT from the environment
                             Cloud Run: no background work after the response returns
systemd                      the plain binary: Type=notify or simple, Restart=always,
                             ExecStart, EnvironmentFile, User=, and a socket unit
                             for socket activation
Lambda                       an adapter; watch cold starts and the frozen execution
                             environment between invocations
```

ROLLOUT & ROLLBACK

```
rolling update               the default; needs backward-compatible schema and API
blue/green                   two full environments, one switch
canary                       a small percentage first, watched by metrics
expand/contract migrations   add column -> backfill -> deploy code -> drop the old
                             one — NEVER a breaking migration with a rolling deploy
feature flags                deploy dark, enable separately, roll back instantly
rollback plan first          if you can't roll back, it isn't a deploy, it's a bet
```

TRAPS & MEMORIZE

```
CGO_ENABLED=1 into scratch    "no such file or directory" for a binary that IS there
missing CA certs              every HTTPS call fails inside scratch
shell-form ENTRYPOINT         signals never reach your process; SIGTERM does nothing
no graceful shutdown          every deploy drops in-flight requests
readiness == liveness         a DB blip restarts every replica simultaneously
liveness that checks the DB   the same outage, self-inflicted
GOMAXPROCS unset in k8s       massive scheduler contention against a 1-CPU limit
no GOMEMLIMIT with a limit    OOMKilled instead of a GC cycle
grace period < drain timeout  the kernel kills you mid-drain
copying source before go.mod  every build re-downloads every dependency
latest tags                   irreproducible builds, and a surprise every Friday
secrets in the image          they're in the layers forever, even if you delete them
no .dockerignore              you just shipped .git to production
breaking migration + rolling deploy   the old pods hit the new schema and crash
```